

UML

Un negozio possiede delle sedi (con un responsabile e un indirizzo) ed è in contatto con dei fornitori. Responsabili e fornitori sono persone, con nome e cognome. Dei responsabili interessa l'anzianità di servizio e dei fornitori la partita iva. Inoltre il negozio rivende prodotti caratterizzati da un codice e da una descrizione. I fornitori forniscono i prodotti a un certo prezzo (fornitori diversi possono fornire lo stesso prodotto a prezzi diversi). Ai prodotti si possono aggiungere e togliere fornitori; per l'aggiunta occorre specificare il prezzo a cui il fornitore vende l'articolo. Al negozio si possono aggiungere e togliere prodotti; per l'aggiunta occorre specificare (tra le altre cose) un fornitore e il prezzo relativo.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente:

Un amministratore invoca il metodo per aggiungere al negozio un nuovo prodotto con codice SM e descrizione “Sapone per le mani”. Il prodotto è fornito dal fornitore SoapSpA (già esistente) al prezzo di €2.49. Il negozio crea il prodotto e invoca il metodo che lo associa al fornitore specificato.

UML

Un ufficio pubblico ha del personale, di cui interessano nome e stipendio. Il personale è suddiviso in impiegati e dirigenti, e l'ufficio è diretto da un dirigente. L'ufficio ha degli sportelli, ciascuno dei quali è servito da un impiegato. Ogni sportello possiede una coda di prenotazioni, ciascuna delle quali riporta il nome del richiedente e l'ora di arrivo. Le prenotazioni sono servite in ordine FIFO. Lo sportello permette di aggiungere una nuova prenotazione in coda e di eliminare quella di testa.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente:

L'utente elimina il primo elemento dalla coda dello sportello 1 che in quel momento contiene due prenotazioni. Successivamente, alle 11:00, l'utente inserisce una nuova prenotazione relativa a Mario Rossi in quella stessa coda. (Tra le due operazioni la coda non viene modificata.) Il metodo di inserzione è *ricorsivo*.

UML

Si vuole progettare un gioco che è costituito da personaggi e livelli.

I personaggi hanno un nome e una immagine che li rappresenta (le immagini sono entità complesse che possono essere visualizzate). Vi sono due tipi di personaggi: protagonisti e nemici. I personaggi possono essere visualizzati e possono agire; *i protagonisti agiscono in modo diverso dai nemici*.

I livelli sono 10. Ogni livello è raffigurato da una immagine e possiede un insieme di personaggi attivi. All'insieme si può chiedere il primo elemento e il successivo elemento (per enumerarli tutti; non è richiesto di mostrare le strutture dati per ottenere l'enumerazione). I livelli permettono di eseguire un turno, cioè di far agire tutti i propri personaggi attivi.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti.

Il gioco chiede a un livello di eseguire un turno. Il livello chiede all'insieme di personaggi attivi un primo elemento, e riceve il protagonista. Il livello fa agire il protagonista. Poi il livello chiede all'insieme di personaggi il prossimo elemento. Riceve null, per cui il turno termina.

UML

Si vuole progettare una applicazione per la gestione di ristoranti che possa trattare piu' di un ristorante (il che è utile per le catene di ristorazione). Ogni ristorante ha un certo numero di tavoli (almeno uno) e gestisce delle prenotazioni. Ogni tavolo possiede un suo numero identificativo e una capienza (cioè il numero di posti). Le prenotazioni contengono giorno, ora, uno o più tavoli, e il cliente che ha prenotato. I clienti hanno nome e numero di telefono, e i ristoranti possono reperire velocemente i clienti usando il loro nome come chiave. Infine ogni ristorante ha una sua lista di attesa (che rispetta l'ordine di arrivo) i cui elementi contengono un cliente e il numero di posti richiesto; un cliente può comparire al massimo in una sola lista di attesa. Si possono inserire clienti in coda alle liste di attesa (specificando anche il numero di posti richiesti); in aggiunta, specificando il numero di posti che si sono liberati, si può cercare nella lista il primo cliente (nell'ordine di arrivo) che ha richiesto quel numero di posti.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti.

L'operatore chiede al ristorante R1 di cercare nella propria lista d'attesa il primo cliente che ha chiesto un tavolo da 2 posti. Il primo elemento nella lista richiede 4 posti, quindi si passa la richiesta al secondo elemento della lista, che contiene una richiesta di due posti. Il cliente corrispondente (C2) viene restituito all'operatore. La scansione della lista avviene in modo *ricorsivo*.

UML

Si vuole modellare una rete ferroviaria. I nodi della rete possono essere solamente stazioni o scambi. Tutti i nodi hanno un nome. Inoltre le stazioni hanno un indirizzo mentre gli scambi hanno una coppia di coordinate (latitudine,longitudine). Gli archi sono etichettati con un numero che rappresenta la lunghezza del tratto ferroviario corrispondente. I percorsi sono liste concatenate di nodi della rete. Uno stesso nodo della rete ferroviaria può appartenere a più percorsi. Ai percorsi si può appendere un nuovo nodo.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle classi che mostra come le classi di associazione vengono trasformate in classi semplici per poter essere implementate.

Esercizio 3: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti. **Se un oggetto ha bisogno di un attributo privato di un altro oggetto dovete mostrare come se lo procura.**

Un utente aggiunge il nodo S1 della rete ferroviaria in coda ad un percorso che già contiene il nodo S2.

UML – Max 13 punti

Si vuole progettare una applicazione gestionale per il mobilificio Poltrone&Cassetti. Ogni negozio Poltrone&Cassetti ha un indirizzo e una lista concatenata di ordini. Ogni ordine ha un numero identificativo e un articolo. Gli articoli hanno una stringa identificativa e possono essere di soli due tipi: interni ed esterni. Quelli interni contengono la fabbrica Poltrone&Cassetti di provenienza, descritta con una stringa. Quelli esterni hanno un fornitore, caratterizzato da nome e indirizzo. Ogni negozio Poltrone&Cassetti fa uso di un database di backend progettato da altri, di cui si sa unicamente che supporta il metodo *query*(String *q*):String. Dai negozi si può eliminare il primo ordine della lista.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.
[max 8 punti]

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti. **Se un oggetto ha bisogno di un attributo privato di un altro oggetto dovete mostrare come se lo procura.**

Un utente rimuove da un negozio N1 il primo ordine O1. La nuova testa della lista di ordini diventa il suo successore O2. L'oggetto O1 viene distrutto.

[max 6 punti]

UML

Si vuole progettare una applicazione gestionale per un ufficio vendite, che possiede degli impiegati caratterizzati da nome, stipendio e ammontare delle vendite. Gli impiegati possono essere venditori o supervisor. Non si possono creare impiegati che non appartengano a una di queste due categorie. I supervisor hanno un bonus; il loro stipendio totale è calcolato sommando l'attributo stipendio con il bonus. Per i venditori lo stipendio coincide con l'attributo omonimo. Un rapporto impiegati è una lista ordinata di impiegati. Un rapporto impiegati è in grado di calcolare la somma delle vendite di tutti i dipendenti che contiene.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti. **Se un oggetto ha bisogno di un attributo privato di un altro oggetto dovete mostrare come se lo procura.**

Un utente chiede di calcolare il totale delle vendite dei dipendenti elencati in un rapporto impiegati che contiene due elementi: un venditore che ha venduto 1500€ di merce e un supervisore le cui vendite ammontano a 1600€. Il calcolo avviene in modo ricorsivo sulla lista di impiegati.

UML

Progettare un sistema di prenotazione viaggi con le seguenti caratteristiche. Un viaggio è una lista concatenata di tratte. Ogni tratta possiede un contratto, rappresentato con una stringa, e un biglietto. I biglietti possono essere solo aerei o ferroviari. Ogni biglietto ha un luogo e una data di partenza e un luogo di arrivo. I biglietti aerei hanno anche un'ora di imbarco, mentre i biglietti ferroviari hanno un codice di sconto rappresentato con una stringa. Ai viaggi si possono aggiungere tratte in coda.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti. **Se un oggetto ha bisogno di un attributo privato di un altro oggetto dovete mostrare come se lo procura.**

Un utente crea una nuova tratta via aerea (che a sua volta crea il proprio biglietto), con contratto standard CS, partenza da Napoli il 15/4/2015 e arrivo a Venezia. Poi appende in modo *ricorsivo* la nuova tratta a un viaggio che già contiene una tratta T0.

UML

Si tratta di realizzare un sito per l'acquisto di libri on line. Il sito può accedere ai libri che distribuisce (caratterizzati da titolo e autore) usando il titolo come chiave. Ogni libro può essere venduto da diverse case editrici (di cui interessano nome e indirizzo), a prezzi diversi.

Il sito possiede dei clienti, di cui mantiene un nickname e una password. Inoltre ogni cliente possiede un carrello della spesa.

Ogni carrello contiene una lista di prodotti, nell'ordine in cui sono stati inseriti. Si può chiedere di calcolare il numero di libri contenuti nel carrello.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti. **Se un oggetto ha bisogno di un attributo privato di un altro oggetto dovete mostrare come se lo procura.**

Un cliente aggiunge al proprio carrello il libro *Mattatoio 5*, di Vonnegut. Il carrello già conteneva il libro *Ben Hur*. Il cliente poi chiede quanti libri contiene il carrello e il calcolo viene svolto con un algoritmo *iterativo* accedendo agli oggetti rilevanti (mostrare i messaggi necessari).

UML

Progettare una applicazione "Calendario". Gli eventi del calendario possono essere solo di due tipi: viaggio e meeting. Tutti gli eventi hanno giorno e ora di inizio e una descrizione. Tutti e tre questi valori sono rappresentati con stringhe. In più i viaggi hanno una durata in giorni, mentre i meeting hanno una durata in minuti. Ogni evento può essere stampato; la stampa è diversa per i due tipi di evento. Il calendario mantiene gli eventi di ciascun (singolo) giorno organizzati in una lista doppiamente concatenata, ordinata rispetto all'ora di inizio. Il calendario può accedere al primo elemento della lista usando il giorno come chiave. Il calendario può stampare gli eventi di un giorno specificato; inoltre può sincronizzare il proprio contenuto con il cloud aziendale mediante il metodo *sync()* che utilizza al suo interno un componente esterno di cui si sa solo che supporta un metodo *syncCloud()*.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti. **Se un oggetto ha bisogno di un attributo privato di un altro oggetto dovete mostrare come se lo procura.**

Il calendario stampa la lista concatenata di eventi che contiene gli eventi E1, E2 ed E3. La scansione della lista avviene in modo *iterativo*.

UML

Si vuole progettare un sistema di avvisi per i monitor di un aeroporto. Comprende dei voli (caratterizzati da un numero di volo, una destinazione, e un orario di partenza rappresentato con una stringa), degli sportelli di check-in (caratterizzati da un numero identificativo e dal numero di terminale in cui si trovano) e dei *gate* (caratterizzati da un numero identificativo e dal terminale in cui si trovano). Inoltre, ogni volo è associato al massimo ad un gate (potrebbe non essere ancora noto) e ad un insieme di sportelli in cui si può fare il check-in. Le schermate informative sono di due soli tipi: *check-in* e *imbarco*. Ogni schermata informativa ha un suo messaggio di intestazione e una lista ordinata di voli, da cui si può eliminare il primo elemento e aggiungerne uno in coda. L'operazione di *refresh*, che aggiorna le schermate, mostra informazioni diverse nelle due tipologie di schermata. Il refresh utilizza un componente esterno – di cui si sa solo che supporta un metodo *get_ads()* – per ricevere le pubblicità da mostrare vicino alle informazioni sui voli.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti. **Se un oggetto ha bisogno di un attributo privato di un altro oggetto dovete mostrare come se lo procura.**

Un operatore chiede a una schermata di aggiungere il volo AX123 alla lista, che al momento contiene un volo. Poi l'operatore fa il refresh della schermata (operazione che causa lo scaricamento delle pubblicità). L'aggiunta è implementata in modo *iterativo*.

UML

Progettare un sistema di supporto al lavoro di ufficio come descritto in seguito. Vi sono dei compiti, ciascuno dei quali ha una descrizione testuale, un responsabile (caratterizzato da nome e email) e una data entro cui il compito deve essere svolto. La data è rappresentata con un intero. Un compito può essere elementare o composito. Quelli compositi rappresentano una sequenza di compiti più semplici; per questo posseggono anche una lista ordinata di compiti (che possono essere elementari o compositi a loro volta). I compiti supportano una operazione di notifica: se la data odierna è maggiore della data di scadenza del compito, allora viene mandata una mail al responsabile per avvisarlo. Il componente che fornisce la data odierna e quello che invia la mail non sono sotto la vostra responsabilità (li sviluppa qualcun altro). Sapete solo che il primo supporta un metodo `get_date()` che restituisce la data, e che il secondo supporta un metodo `send_mail(...)` i cui input sono il destinatario e il testo del messaggio.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti. **Se un oggetto ha bisogno di un attributo privato di un altro oggetto dovete mostrare come se lo procura.**

L'utente invoca il metodo di notifica sul compito elementare C0, che ha scadenza D0. La data odierna è $D > D0$. Il metodo di notifica legge la data odierna mediante l'apposito componente; poichè è maggiore di D0, il metodo di notifica invia la mail al responsabile di C0 mediante l'apposito componente.

UML

Progettare una applicazione per la gestione di utenze telefoniche con le seguenti caratteristiche. Ogni utente ha un nome e un credito. Inoltre è associato a due liste concatenate di telefonate, quelle in ingresso e quelle in uscita. Tutte le telefonate hanno una durata. In aggiunta, quelle in ingresso hanno il numero chiamante, mentre quelle in uscita hanno il numero chiamato e un costo. Non si possono creare telefonate generiche (ma solo in ingresso o in uscita). Ogni utente è associato a un contratto che specifica le date di inizio e fine del contratto stesso, e il costo per minuto delle chiamate.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti.

Un utente aggiunge in coda alla lista di telefonate in uscita una nuova chiamata al numero 333-456789 della durata di 50 secondi. Prima dell'aggiunta la lista contiene già due telefonate. L'aggiunta deve essere eseguita in modo *iterativo*.

UML

Una fabbrica produce un certo numero di prodotti caratterizzati da un codice e dal prezzo. La fabbrica permette l'inserimento di ordinativi, ciascuno dei quali specifica uno dei prodotti, la quantità richiesta, il cliente che la chiede e l'impiegato che segue l'ordine. La fabbrica permette anche la cancellazione dell'ordinativo più vecchio quando questo è stato completato. Gli ordinativi vengono smaltiti in ordine di arrivo (FIFO) e sono conservati in una lista concatenata. I clienti hanno un nome, un codice fiscale e una partita iva. Gli impiegati hanno un nome, un codice fiscale e uno stipendio.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente:

L'utente inserisce un ordinativo O3 in un momento in cui sono in lavorazione altri due ordinativi, O1 e O2. La creazione dell'oggetto O3 è responsabilità del metodo di inserimento, mentre il prodotto, il cliente e l'impiegato esistono già. L'inserzione è *iterativa*. Successivamente l'utente elimina il primo ordinativo (O1). Il metodo di eliminazione rimuove O1 dalla lista e lo distrugge.

UML

Progettare un sistema per i trasporti locali. L'azienda possiede un insieme di veicoli che possono essere solamente bus, traghetti o tram. Tutti i veicoli hanno una capienza (numero massimo di passeggeri) e un metodo di stampa che fa cose diverse per i diversi tipi di veicoli. Inoltre l'azienda ha dei dipendenti, con nome e stipendio. L'azienda mantiene una lista ordinata di turni, ciascuno dei quali è specificato da: un orario di inizio, uno di fine (rappresentati con stringhe), un dipendente e il veicolo che pilota. Inoltre si possono contare quanti turni sono nella lista

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti. **Se un oggetto ha bisogno di un attributo privato di un altro oggetto dovete mostrare come se lo procura.**

L'azienda invia alla lista dei turni, che ne contiene tre, la richiesta di contare i propri membri. ATTENZIONE: L'implementazione deve essere iterativa (non ricorsiva).

UML

Si vuole progettare un meccanismo di conference call (teleconferenza) per il client di un sistema VOIP. Il client possiede un insieme di contatti, caratterizzati da un id e da un nome, che il client può recuperare usando il nome come chiave. Il client può in ogni momento essere associato con al massimo una conference call. Una conference call possiede una lista concatenata di partecipanti, mantenuta nell'ordine in cui questi sono aggiunti alla teleconferenza. Ogni partecipante è un contatto. Il client utilizza un componente esterno per effettuare la connessione vocale, di cui si sa solo che supporta i metodi *call* e *close*, che prendono un contatto in input e iniziano e terminano la chiamata con quel contatto, rispettivamente.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente:

L'utente chiede al client di creare una conference call (inizialmente vuota). Poi chiede al client il contatto di Mario e lo aggiunge alla lista di partecipanti della conference call.

UML

Progettare un sistema per il controllo del traffico aereo. Un piano di volo è una lista concatenata di punti di passaggio detti *waypoint*. Alcuni casi particolari di *waypoint* sono coordinate e aeroporti. Le coordinate sono coppie (*latitudine*, *longitudine*), mentre gli aeroporti sono caratterizzati da un nome e da una coordinata.

Deve essere sviluppato un componente detto “Database” che può accedere agli aeroporti usando il nome come chiave e supporta un metodo *search* che data una stringa restituisce l’aeroporto il cui nome è la stringa data.

Ai piani di volo si possono accodare nuovi *waypoint*. Nel caso degli aeroporti, l’aggiunta può essere effettuata anche fornendo solo il nome (questo metodo sfrutta il database per recuperare l’aeroporto).

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente:

Un pilota (utente) aggiunge l’aeroporto di Linate (fornendo la stringa “Linate”) a un piano di volo che già contiene gli aeroporti di Capodichino e Roma. Il metodo di aggiunta ottiene Linate chiamando *search*, poi lo inserisce in coda al piano di volo. L’inserzione è ricorsiva.

UML

Si vuole progettare un sistema di gestione per la piscina del CUS. La piscina ha degli utenti, a cui può accedere usando il nome come chiave. Oltre al nome, ad ogni utente è associata una lista concatenata di ingressi (caratterizzati da data e ora). Agli utenti si può aggiungere un ingresso, che verrà posto in coda alla lista dell'utente. Alcuni utenti sono universitari; questi posseggono anche un numero di matricola. Gli utenti implementano l'interfaccia *Serializable* che ha due metodi: *readObject()* e *writeObject()*.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 3: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti. **Se un oggetto ha bisogno di un attributo privato di un altro oggetto dovete mostrare come se lo procura.**

Un amministratore aggiunge l'ingresso con data D1 e ora O1 a un utente che nella sua lista ha già due ingressi precedenti. L'inserimento in coda alla lista è *iterativo*.

UML

Si vuole progettare una applicazione gestionale per una azienda, che possiede degli impiegati caratterizzati da nome e stipendio. Gli impiegati possono essere tecnici, amministrativi, o manager. Non si possono creare impiegati che non appartengano a una di queste tre categorie. I manager hanno un bonus; il loro stipendio totale è calcolato sommando l'attributo stipendio con il bonus. Per gli altri tipi di impiegato lo stipendio coincide con l'attributo omonimo. Un *rapporto impiegati* è una lista ordinata di impiegati. Un rapporto impiegati è in grado di calcolare la somma degli stipendi totali di tutti i dipendenti che contiene.

Esercizio 1: Disegnare un diagramma delle classi per queste specifiche.

Esercizio 2: Disegnare un diagramma delle sequenze per lo scenario seguente, mostrando tutte le interazioni tra gli oggetti coinvolti.

Un utente chiede di calcolare il totale degli stipendi dei dipendenti elencati in un rapporto impiegati che contiene due elementi: un tecnico con stipendio 1500, e un manager con stipendio 2000 e bonus 500. Il calcolo avviene in modo *iterativo* sulla lista di impiegati.